

Tema 1 — Modelo de cómputo, exclusión mutua y modelo de memoria

Fuentes: Teórica 01-Mutex, Práctica clase-01, Guía 1. **Ejercicio 1 del parcial.**

1. Cómo modelamos un programa concurrente

- **Programa secuencial:** una función de estado a estado. Mismo input, mismo output (determinismo).
- **Programa concurrente:** varios threads cuyas instrucciones se **entrelazan** (*interleaving*) en cualquier orden que respete el orden **interno** de cada thread.
 - El resultado **no es determinístico**: la misma entrada puede dar salidas distintas.
- **Instrucción atómica:** se ejecuta "de un saque", nadie se mete en el medio.
 - Por defecto asumimos que son atómicas **las lecturas y escrituras de variables** y las cuentas con variables locales.
 - `x = x + 1` **NO es atómica**. Son 3 pasos:

```
tmp = x      // leo la compartida
tmp = tmp + 1 // cuenta local
x = tmp      // escribo la compartida
```

Si dos threads leen `x = 0` antes de que alguno escriba, los dos escriben 1: **se perdió un incremento**. Esto es el ejemplo base de *race condition*.

- **Independencia del tiempo:** jamás podés razonar "este thread es más rápido" o "`sleep(1000)` asegura que el otro termina antes". El scheduler puede cortar en cualquier instrucción.

Estado, traza, grafo

- **Estado:** el valor de las variables + en qué instrucción está cada thread (su "program counter", PC).
- **Traza:** una ejecución concreta, un camino en el grafo de estados. En el parcial se escribe como tabla:

T1	T2	Estado
<code>tmp = x</code>		<code>T1.tmp = 0</code>
	<code>tmp = x</code>	<code>T2.tmp = 0</code>
<code>x = tmp + 1</code>		<code>x = 1</code>
	<code>x = tmp + 1</code>	<code>x = 1</code> ← se perdió uno

- **Diagrama de transición de estados:** todos los estados posibles y cómo se pasa de uno a otro. Crece exponencialmente, así que solo sirve para programas chiquitos (Guía 1, ej 1–4).

2. Qué significa "correcto"

- **Safety** = algo malo **nunca** pasa (vale en *todo* estado). Ejemplo: "nunca hay dos en la sección crítica".
- **Liveness** = algo bueno **tarde o temprano** pasa. Ejemplo: "si quiero entrar, en algún momento entro".
- Un programa que no hace nada es *safe* pero inútil. Lo difícil es conseguir las dos cosas.
- **Fairness (débil)**: asumimos que toda instrucción que queda **continuamente habilitada** termina ejecutándose. Sirve para descartar trazas absurdas donde un thread nunca corre.
 - **Importante**: cuando mostrás inanición, tu traza infinita tiene que ser *fair*.

3. El problema de la exclusión mutua

Cada thread repite: sección no crítica (SNC) → pre-protocolo → SECCIÓN CRÍTICA (SC) → post-protocolo .

Supuestos:

- La **SC siempre termina**.
- La **SNC puede no terminar** (un thread se puede quedar ahí para siempre).
- No se comparten variables entre la SC y la SNC.

Hay que garantizar **tres cosas**:

1. **Exclusión mutua** (safety): nunca hay 2 en la SC.
2. **Ausencia de deadlock** (liveness): si varios quieren entrar, **alguno** entra.
3. **Ausencia de inanición** (liveness): **todo** el que quiere entrar, entra. La cátedra también la llama **Garantía de Entrada (GdE)**.

En criollo: es un baño con una sola llave. (1) Nunca hay dos adentro. (2) Si hay cola, alguien entra. (3) Nadie se queda en la cola para siempre.

4. Los algoritmos clásicos, línea por línea

4.0 Qué intentan hacer todos

Todos quieren construir el "candado" de la sección crítica **usando solo lecturas y escrituras de variables compartidas**. No hay instrucciones especiales ni sistema operativo. Cada algoritmo es un **pre-protocolo** (lo que hacés antes de entrar a la SC) y un **post-protocolo** (lo que hacés al salir).

La teórica los presenta como una escalera. Cada intento arregla el problema del anterior y rompe otra cosa, hasta llegar a los que funcionan:

Intento I	(una bandera compartida)	→ rompe MUTEX
Intento II	(una bandera por thread)	→ arregla mutex, rompe DEADLOCK
Intento III	(turno)	→ arregla los dos, rompe INANICIÓN

Dekker	(II + III)	→ funciona (2 threads)
Peterson	(II + III, más simple)	→ funciona (2 threads)
Bakery	(números de turno)	→ funciona (N threads)

Notación: hay dos threads con `id` 0 y 1, y `otro = (id + 1) % 2` es el id del otro. `while (cond);` con punto y coma significa "quedate girando mientras `cond` sea verdadera" (busy-waiting).

4.1 Intento I — "levanto la bandera" (una sola bandera para todos)

```
global flag = false           // true = "hay alguien adentro"

while (flag);                 // (1) espero mientras haya alguien adentro
flag = true;                  // (2) marco que ahora estoy yo
    // SECCIÓN CRÍTICA
flag = false;                 // (3) salgo: libero
```

Idea: la bandera es un cartel de "ocupado" en la puerta del baño. Miro el cartel; si dice libre, lo pongo en ocupado y entro.

Qué falla: exclusión mutua. Mirar el cartel (1) y darlo vuelta (2) son **dos pasos separados**, y el otro se puede meter en el medio (*check-then-act*):

T0	T1	flag
<code>while(flag) : ve false, sale del while</code>		false
	<code>while(flag) : ve false, sale del while</code>	false
<code>flag = true</code>		true
	<code>flag = true</code>	true
en la SC	en la SC	← los dos adentro

Deadlock no hay: si los dos quieren, alguno pasa. Pero el mutex se rompe, así que no sirve.

4.2 Intento II — "levanto MI bandera" (una bandera por thread, marco antes de mirar)

```
global flag[2] = {false, false} // flag[i] = "el thread i quiere entrar (o está adentro)"

flag[id] = true;                 // (1) PRIMERO aviso que quiero entrar
while (flag[otro]);              // (2) DESPUÉS espero mientras el otro también quiera
    // SECCIÓN CRÍTICA
flag[id] = false;               // (3) salgo: ya no quiero
```

Idea: para arreglar el intento I, se invierte el orden: primero anuncio mis intenciones y recién después miro al otro. Ahora hay una bandera **por thread** (cada uno escribe solo la suya).

Mutex: sí. Si T0 entró es porque, **con su bandera ya levantada**, vio `flag[1] == false`. Para que T1 entre tendría que ver `flag[0] == false`, pero T0 la tiene en true hasta salir. Imposible.

Qué falla: deadlock (técnicamente livelock). Los dos son "demasiado educados":

T0	T1	flag
<code>flag[0] = true</code>		{T, F}
	<code>flag[1] = true</code>	{T, T}
<code>while(flag[1]) : gira</code>	<code>while(flag[0]) : gira</code>	← para siempre

Es un *livelock* porque siguen ejecutando (girando) pero nadie progresa. En el parcial se lo trata como una falla de "ausencia de deadlock".

4.3 Intento III — "uso un turno"

```
global turno = 0           // de quién es el turno

while (turno != id);       // (1) espero hasta que sea MI turno
    // SECCIÓN CRÍTICA
turno = otro;              // (2) al salir, le paso el turno al otro
```

Idea: en vez de banderas, una sola variable dice quién puede entrar. Es como una pelota que se van pasando.

- **Mutex: sí.** `turno` tiene un solo valor a la vez, así que solo uno puede pasar el while.
- **Deadlock: no.** `turno` siempre vale 0 o 1, así que uno de los dos siempre puede pasar.
- **Qué falla: inanición.** Obliga a **alternar estrictamente** (0, 1, 0, 1, ...). Si T1 se queda en su SNC para siempre (algo permitido), nunca devuelve el turno, y T0 no puede volver a entrar aunque la SC esté vacía. El problema no se ve si en el análisis borrás la SNC; la teórica justamente advierte eso.

4.4 Dekker — II + III: banderas, y el turno solo para desempatar

```
global flag[2] = {false, false}
global turno = 0           // acá "turno" = quién tiene PRIORIDAD si hay empate

flag[id] = true;           // (1) quiero entrar (como en II)
while (flag[otro]) {       // (2) mientras el otro TAMBIÉN quiera (conflicto)...
    if (turno == otro) {   // (3) ...y la prioridad sea suya:
        flag[id] = false; // (4) me bajo: bajo mi bandera para dejarlo pasar
        while (turno != id); // (5) espero a que me devuelva la prioridad
        flag[id] = true;  // (6) vuelvo a pedir
    }                      // (si la prioridad es mía, sigo girando en (2) con mi bandera
}                          // levantada: el otro se va a bajar en su (4))
```

```
// SECCIÓN CRÍTICA
turno = otro;           // (7) al salir, la prioridad pasa al otro
flag[id] = false;       // (8) ya no quiero
```

Idea: usar las banderas del intento II (así no hace falta alternar), y el turno del intento III **solo para desempatar** cuando los dos quieren a la vez.

- **Sin conflicto:** si el otro no quiere (`flag[otro] == false`), paso directo por (2) sin mirar el turno. Por eso **no hay inanición por la SNC** como en el intento III.
- **Con conflicto** (los dos levantaron la bandera): el que **no** tiene la prioridad se baja (4), espera (5) y vuelve a intentar (6). El que la tiene mantiene su bandera y espera a que el otro la baje. Así se rompe el deadlock del intento II.
- **Sin inanición:** al salir de la SC, el que entró le cede la prioridad al otro (7). Si los dos vuelven a competir, gana el otro.
- **Contra:** es enredado (loop dentro de loop) y solo sirve para 2 threads.

4.5 Peterson — la versión simple y elegante de lo mismo

```
global flag[2] = {false, false}
global turno = 0

flag[id] = true;           // (1) quiero entrar
turno = otro;              // (2) pero si hay empate, pasá vos (te cedo)
while (flag[otro] && turno == otro); // (3) espero SOLO si el otro quiere Y la cortesía la hice yo último
    // SECCIÓN CRÍTICA
flag[id] = false;         // (4) ya no quiero
```

Idea en criollo: llegás a una puerta al mismo tiempo que otra persona. Decís "quiero pasar" (flag) y enseguida "pero pasá vos" (turno = otro). Si los dos dicen "pasá vos", **el que lo dijo último es el que espera**: su "pasá vos" pisó al del otro.

Por qué funciona, caso por caso:

- **El otro no quiere** (`flag[otro] == false`): la condición de (3) es falsa y paso directo. No hay alternancia forzada; eso arregla el intento III.
- **Los dos quieren a la vez:** los dos escriben `turno`, pero `turno` termina con **un solo valor**, el del último que escribió. Supongamos que T0 escribió `turno = 1` y después T1 escribió `turno = 0`:
 - T0 evalúa `flag[1] && turno == 1` → `true && false` → **entra**.
 - T1 evalúa `flag[0] && turno == 0` → `true && true` → **espera**.

El último en ceder espera, así que nunca esperan los dos. Eso arregla el deadlock del intento II.

- **Sin inanición:** si T0 sale y quiere volver a entrar enseguida, en (2) escribe `turno = 1` **él mismo**. Entonces T1, que estaba esperando, ahora ve `turno == 1` y pasa. T0 le puede ganar a T1 **a lo sumo una vez** (espera acotada).

Qué lo diferencia de Dekker: mismos ingredientes (banderas + turno), pero la cortesía se hace **al entrar** (`turno = otro` antes del while) y no al salir. Además no hace falta bajar y volver a subir la bandera, así que no hay loop anidado. Es más corto y más fácil de probar.

⚠ **El orden de (1) y (2) importa.** Si primero hacés `turno = otro` y después `flag[id] = true`, se rompe el mutex: el otro puede ver tu bandera todavía en false, entrar, y después vos también pasás. Esto aparece mucho en los ejercicios del tipo "¿qué pasa si cambio el orden de estas dos líneas?".

⚠ **Solo sirve para 2 threads.** La Guía 1 ej 11 lo generaliza a n con `otro = (id+1) % n`, y ahí se rompe el mutex: cada thread mira a un solo vecino, no a todos.

4.6 Bakery (panadería) — para N threads

```
global entrando[N] = {false, ...} // entrando[i] = "i está sacando número ahora mismo"
global numero[N]   = {0, ...}    // numero[i] = número de i; 0 = "no quiero entrar"

entrando[id] = true;                // (1) aviso: estoy sacando número
numero[id] = 1 + max(numero[0..N-1]); // (2) saco un número más grande que todos los que veo
entrando[id] = false;              // (3) terminé de sacar número
for (j = 0; j < N; j++) {          // (4) reviso a cada uno de los otros:
    while (entrando[j]);           // (5) si j está sacando número, espero a que
    termine
    while (numero[j] != 0 &&        // (6) espero mientras j quiera entrar
           (numero[j] < numero[id] || // y tenga un número menor que el mío
            (numero[j] == numero[id] && j < id))); // o el mismo número pero menor id
}
// SECCIÓN CRÍTICA
numero[id] = 0;                    // (7) salgo: tiro mi número
```

Idea en criollo: la panadería. Al llegar sacás un número mayor que todos los que ves. Atienden en orden de número; el que tiene el número más chico entre los que esperan, pasa.

Qué hace cada parte y por qué:

- **(2) `1 + max`**: garantiza que el que llega después saca un número más grande que los que ya estaban esperando. Por eso es **FIFO** (orden de llegada) y **no hay inanición**.
- **Empates `(numero[j] == numero[id] && j < id)`**: el `max` **no es atómico** (lee N variables de a una). Dos threads pueden calcularlo a la vez, ver los mismos valores y **sacar el mismo número**. El empate se resuelve por id: gana el id menor. Esto hace que el orden sea total, sin empates: se compara el par (número, id). La Guía 1 ej 13 saca este desempate y pregunta qué pasa (pista: los dos se quedan esperándose).

- (5) `while (entrando[j]) ("la puerta")`: es la parte más sutil. Sin ella puede pasar esto:
 1. j está calculando su `max` (paso 2) y todavía no escribió su número (`numero[j]` sigue en 0).
 2. Yo miro `numero[j] == 0`, pienso "j no quiere entrar", lo paso y entro a la SC.
 3. j termina de escribir un número **igual al mío** (los dos leímos lo mismo) pero con **id menor**.
 4. j me revisa a mí, ve que tiene prioridad sobre mí, y **también entra**. Mutex roto.

Con `entrando[j]`, antes de comparar números espero a que j termine de sacar el suyo, así que nunca comparo contra un número "a medio escribir".

- (6) `numero[j] != 0`: si j tiene 0, no quiere entrar, así que no lo espero.
- (7) `numero[id] = 0`: al salir aviso que ya no compito.

Qué lo diferencia de los demás:

- Funciona para **N threads** (Dekker y Peterson solo para 2).
- No necesita que ninguna escritura sea atómica respecto de las demás: cada thread escribe **solo sus propias** variables. Todo lo compartido se lee.
- **Contras**: los números crecen sin límite (si siempre hay alguien esperando, nunca vuelven a 0), y para entrar hay que leer las 2N variables.

4.7 Resumen comparativo

	Variables	Mutex	Sin deadlock	Sin inanición	Threads	Idea clave
I	1 flag	X	✓	X	2	chequeo y después marco → se cuelan
II	flag[2]	✓	X	X	2	marco y después chequeo → los dos esperan
III	turno	✓	✓	X	2	alternancia forzada
Dekker	flag[2] + turno	✓	✓	✓	2	turno solo para desempatar; se cede al salir
Peterson	flag[2] + turno	✓	✓	✓	2	"pasá vos" al entrar; espera el último que cedió
Bakery	entrando[N] + numero[N]	✓	✓	✓	N	número de orden + desempate por id + "puerta"

4.8 Con instrucciones atómicas de hardware (mucho más fácil)

Si el hardware da una instrucción que **lee y escribe en un solo paso atómico**, el problema del intento I (mirar y marcar por separado) desaparece.

```
// TEST-AND-SET: devuelve el valor viejo y deja 1, todo atómico
global comp = 0
repeat
    local = test-and-set(comp)    // leo si estaba ocupado Y lo marco ocupado, en un paso
until (local == 0)                // si estaba en 0, era libre y ahora es mío
    // SECCIÓN CRÍTICA
comp = 0                          // libero
```

Es el intento 1 pero con los pasos (1) y (2) fusionados en uno. **Mutex y deadlock-free: sí. Sin inanición: no:** cuando se libera, gana el que ejecute primero su TAS, y uno puede perder siempre. **exchange** hace lo mismo (intercambia un 1 local con la compartida).

```
// COMPARE-AND-SWAP: "si vale lo que espero, cambialo", atómico
while (CAS(comp, false, true) != false);    // si estaba libre (false), lo pongo en true; si no,
reintento
    // SECCIÓN CRÍTICA
comp = false
```

Mismas propiedades que TAS. El CAS es la base de todo lo lock-free del tema 4.

```
// FETCH-AND-ADD: sistema de tickets (como sacar número en el banco)
global ticket = 0, turno = 0
miTurno = fetch-and-add(ticket, 1)    // saco número: me quedo con el actual y lo incremento,
atómico
while (turno != miTurno);              // espero a que llamen a mi número
    // SECCIÓN CRÍTICA
fetch-and-add(turno, 1)                // al salir, llaman al siguiente número
```

Es Bakery hecho fácil: como sacar número es atómico, **no hay empates** ni hace falta la "puerta". Además **no hay inanición**: se atiende en orden FIFO estricto.

- Guía 1 ej 14: la versión mal hecha **decrementa el ticket** al salir en vez de avanzar el turno. Pensá qué pasa si otro saca número justo después.

Todas estas soluciones hacen **busy-waiting** (gastan CPU girando en un `while`). Los semáforos (tema 2) arreglan eso.

4.9 Cómo se prueba (o se refuta) cada propiedad — recetas

Regla de oro:

- Para decir que una propiedad **NO vale** alcanza con **una traza** (un contraejemplo concreto).
- Para decir que **SÍ vale**, una traza no sirve: tenés que argumentar para **todas** las ejecuciones. Casi siempre se hace **por el absurdo** o con un **invariante**.

Mutex (safety): "nunca hay dos en la SC"

Para romperlo: buscá el hueco entre "miro" y "marco" (*check-then-act*), o partí un `x++` en `tmp = x; x = tmp + 1`, y armá una traza que termine con **los dos en la SC**.

Para probarlo, por el absurdo (es la forma más común):

"Supongamos que T0 y T1 están los dos en la SC al mismo tiempo. Miremos **cuál de los dos pasó último** su `while` de espera (o cuál escribió último una variable clave). Cuando ese pasó el `while`, el otro ya había hecho X (levantado su bandera, escrito el turno...), así que la condición del `while` tendría que haber sido verdadera y no podría haber pasado. Absurdo."

- **El truco:** ordenar en el tiempo las escrituras y las lecturas. Las preguntas son "¿quién escribió último?" y "¿qué vio cuando leyó?".
- **Ejemplo, Peterson:** supongamos que los dos están en la SC. Los dos escribieron `turno`; digamos que **T1 fue el último** (quedó `turno = 0`).
 - Cuando T1 evaluó su `while`, eso fue después de su propia escritura de `turno`, así que vio `turno == 0`.
 - Además vio `flag[0] == true`, porque T0 la levantó antes de escribir `turno`, y la escritura de T0 fue antes que la de T1.
 - Entonces `flag[0] && turno == 0` era verdadera, y T1 no podía pasar. Absurdo.

Para probarlo, con un invariante: una propiedad que vale al principio y que **cada paso conserva** (inducción sobre la traza). De ahí deducís `#enSC ≤ 1`.

- Con semáforos: `#enSC + S.V = 1`, y como `S.V ≥ 0`, sale `#enSC ≤ 1`.
- Con turno: "si Ti está en la SC, entonces `turno == i`", y `turno` tiene un solo valor.

Ausencia de deadlock (liveness): "si varios quieren entrar, alguno entra"

Para probarlo, por el absurdo (esta es la frase que buscabas):

"Supongamos que hay deadlock: todos los que quieren entrar están **trabados en su espera para siempre y ninguno está en la SC**. Entonces, a partir de cierto momento, **nadie escribe nada** (todos están girando en un `while` que solo lee, o dormidos en un `acquire`), así que las variables quedan **congeladas**. Con esos valores fijos, las condiciones de espera tendrían que ser **todas verdaderas a la vez**. Mostramos que eso es imposible."

- **Peterson:** si los dos están trabados, valen `flag[1] && turno == 1` y `flag[0] && turno == 0`. Eso da `turno == 1` y `turno == 0` a la vez. Absurdo.
- **Semáforo mutex:** si todos están bloqueados, `S.V = 0` y `#enSC = 0`, así que `#enSC + S.V = 0 ≠ 1`. Absurdo.
- **Varios recursos tomados en orden global** (filósofos, cuentas del banco): si hubiera deadlock, habría un **ciclo** de esperas: A espera algo que tiene B, B espera algo que tiene C, ..., y el último espera algo que tiene A. Pero cada uno espera un recurso de **número mayor** a los que tiene, así que recorriendo el ciclo los números crecen siempre y volvés al principio. Absurdo.

- **Monitores:** mostrás que todo thread que hace `wait` tiene a alguien que tarde o temprano cambia la condición y hace `signal`.

Para romperlo: una traza que llegue a un estado donde todos esperan, y mostrás que desde ahí ninguna condición cambia nunca (las variables están congeladas y las condiciones de espera son todas verdaderas).

- Ejemplo típico: los dos levantan su bandera antes de mirar (intento II), o cada uno toma un recurso y espera el del otro (filósofos).

Ausencia de inanición / garantía de entrada (liveness): "todo el que quiere, entra"

Para probarlo, por el absurdo:

"Supongamos que T0 quiere entrar y queda esperando **para siempre**. Como no hay deadlock (ya lo probé), los otros sí van entrando y saliendo (la SC siempre termina). Miro qué hace el otro después de salir. Hay dos casos:

- **(a)** se queda en su SNC: entonces su bandera queda abajo y T0 pasa;
- **(b)** vuelve a pedir entrar: en ese caso, al pedir, **él mismo** hace algo que destraba a T0 (le cede el turno, saca un número mayor...).

En los dos casos la condición que traba a T0 se vuelve falsa y **se queda falsa**, porque solo T0 podría volver a cambiarla y T0 está esperando. Absurdo."

- **Ejemplo, Peterson:** si T1 vuelve a pedir, escribe `turno = 0`. Desde ahí, T0 ve `turno == 0` para siempre (solo T0 escribiría `turno = 1`, y está esperando), así que T0 pasa.
- **No te olvides del caso "el otro se queda en la SNC para siempre"**. Justamente así se cae el intento III.
- **Espera acotada (bounded waiting):** una forma más fuerte y cómoda. **Contás cuántas veces te pueden pasar adelante** y mostrás que es un número finito.
 - Peterson: 1 vez.
 - Semáforo fuerte con N threads: $N-1$.
 - Bakery o ticket: solo los que ya tenían un número menor que el tuyo, que son finitos.
- **Con semáforos débiles:** la respuesta suele ser **"no hay garantía"**, porque el release puede despertar a cualquiera y los que recién llegan se pueden colar.

Para romperlo: una **traza infinita** donde T0 quiere entrar y **nunca** entra.

- La forma práctica: mostrá un **ciclo**. Llegás a un estado, pasan cosas (el otro entra y sale; T0 sí ejecuta su chequeo pero justo ve la condición que lo traba), y **volvés al mismo estado**. Repetir ese ciclo para siempre da la traza infinita.
- ⚠ La traza tiene que ser **fair**: T0 tiene que ejecutar sus pasos, no vale "T0 nunca corre". La gracia es que corre, pero **siempre mira en el peor momento**.
- **Ejemplos:**
 - Semáforo débil con 3 threads: p y r se pasan el permiso y q nunca lo recibe.

- Turno (intento III): el otro se queda en la SNC.
- Elegir por menor id: 0 y 1 se turnan y 2 nunca entra.

Tabla resumen

Propiedad	Para ver que FALLA	Para ver que VALE
Mutex	traza que termina con 2 en la SC	absurdo "los dos adentro → el último que entró no podía pasar", o invariante
Sin deadlock	traza a un estado donde todos esperan, con las variables congeladas	absurdo "todos trabados, nadie en la SC, variables fijas → las condiciones no pueden ser todas verdaderas", o absurdo de ciclo con orden global
Sin inanición	traza infinita y fair (un ciclo) donde uno nunca entra	absurdo "T0 espera para siempre → lo que hace el otro al salir o al volver a pedir lo destraba", o cota de sobrepasos

5. Modelo de memoria (práctica 1, segunda mitad) — va en el inciso (c) del parcial

- Todo lo anterior supone **Consistencia Secuencial (SC)**: las instrucciones de todos se ejecutan en *algún* orden total que respeta el orden de cada thread.
- **El hardware real no cumple SC.**
 - **Store buffer**: cada core escribe primero en una colita privada y sigue de largo. Los demás cores todavía no ven esa escritura.
 - En **x86 (TSO)** esto permite que un *load* posterior "se adelante" a un *store* anterior (reordenamiento Store→Load).
 - **Dekker litmus**: `T1: x = 1; r1 = y`, `T2: y = 1; r2 = x`. Bajo SC es imposible que `r1 == r2 == 0`, pero en x86 real **pasa**.
 - **ARM/POWER** reordenan todavía más cosas.
- **FENCE** (barrera de memoria, `mfence`): "no sigas hasta vaciar el store buffer". Se pone entre el store y el load.
- **Instrucciones RMW atómicas** (`xchg`, `lock cmpxchg`, `lock xadd`): además de ser atómicas, en x86 ordenan la memoria.
- **El compilador también reordena**, o te guarda una variable en un registro. Un `while(!flag);` puede no enterarse nunca de que `flag` cambió.
- **Java Memory Model (JMM)**:
 - Una **data race** son dos accesos a la misma variable, al menos uno escritura, sin sincronización que los ordene.
 - **DRF-SC**: si tu programa **no tiene data races**, se comporta como si fuera SC.
 - **volatile**: una escritura volatile *happens-before* toda lectura posterior de esa variable. Da **visibilidad** y **prohíbe reordenar**. **No da atomicidad**: `count++` con `volatile` sigue mal.
 - Ojo: `volatile int[] a` hace volátil la **referencia**, no los elementos → usar `AtomicIntegerArray`.

- **Idea para el parcial:** toda ejecución SC es también una ejecución válida en TSO/Java. Entonces, si algo **ya falla bajo SC**, sigue fallando en hardware real, y además pueden aparecer fallas nuevas.

6. Procesos vs threads (práctica 1, cultura general, poco probable en el parcial)

- **Proceso:** contenedor de recursos (espacio de memoria, archivos).
- **Thread:** lo que ejecuta. Tiene su propio stack, PC y registros, y **comparte la memoria** con los demás threads del proceso.
- **Concurrencia** (progreso intercalado, puede ser en 1 core) vs **paralelismo** (literalmente al mismo tiempo, en varios cores).
- **Modelos de mapeo** de threads de usuario a threads del kernel:
 - **N:1:** baratos, pero sin paralelismo.
 - **1:1:** los `Thread` clásicos de Java/pthreads.
 - **N:M:** los *virtual threads* de Java.

7. Guía 1 — qué entrena cada ejercicio

Ej	Concepto
1–4	semántica, diagramas de estados, contar trazas (baja prioridad)
5	carreras sobre una variable <code>found</code> (¿quién la pisa?)
6–8	razonar sobre interleavings: ¿qué salidas son posibles?, ¿puede no terminar?
9	bakery para 2 "simplificado", ¿resuelve mutex?
10	tickets con <code>PedirTurno</code> / <code>LiberarTurno</code> : mismo formato que el simulacro (a: no atómico, b: atómico)
11	Peterson generalizado a $n > 2$
12	<code>algunVerdadero</code> no atómico vs atómico
13	bakery sin el desempate <code>j < id</code>
14	fetch-and-add mal usado; hay que arreglarlo
15	<code>tomarFlag</code> atómico vs no atómico

Los ej 10–15 son el formato exacto del Ej 1 del parcial. Para cada uno: revisá las 3 propiedades y, para cada propiedad que falle, dá una traza.